Week 1 Interface Notes

Display

The framebuffer starts at 0x20000000 and uses one byte per pixel. That byte isn't a color. It's an index into a 256-entry palette at 0x21000000, where each entry is 0x00RRGGBB. To get the address of a pixel we use FB + y*W + x, counting from the top left corner.

The default screen is 192 x 144. The Resolution button can change it anywhere from 16 to 640 wide and 16 to 480 tall, and applying a new size resets the CPU. A program can read the current size with lw from 0x21001000 (width) and 0x21001004 (height). Those two addresses aren't in the reference panel on the page, only in the source code, so we checked there.

Storing to the framebuffer doesn't change what you see by itself. The monitor only updates when the program writes anything to 0x21001008 (PRESENT). There's also a clear register at 0x2100100c. Write a color index to it and the whole screen fills with that color.

The default palette starts with 0 black, 1 white, 2 red, 3 green, 4 blue, 5 yellow, 6 magenta, 7 cyan, 8 dark grey and 9 light grey.

Input

Every input is read with lw from the address below. The screen has to have focus first, so click it or press Run.

Input	Address	What we read
Mouse x	0x40000030	x position in framebuffer pixels, already limited to 0 through W-1
Mouse y	0x40000034	y position in framebuffer pixels, already limited to 0 through H-1
Mouse buttons	0x40000038	bit 0 is the left button, 1 while held and 0 after release
Key held	0x40000028	ASCII of the key that is down right now, or 0 if none
Key buffer	0x4000002c	the next key press, or 0 if none; reading it removes it from the buffer

Only the left mouse button is reported. The arrow keys read as 37 (left), 38 (up), 39 (right) and 40 (down). Letters keep their case, so c is 99 and C is 67. Space is 32, Enter is 13, Backspace is 8, Escape is 27 and Tab is 9.

The difference between the two key registers matters. The held register keeps returning the same value for as long as the key is down, so it's better for something like moving a cursor. The buffer register gives each key press once, so it's better for a single action like clear. The buffer holds the last 64 presses.

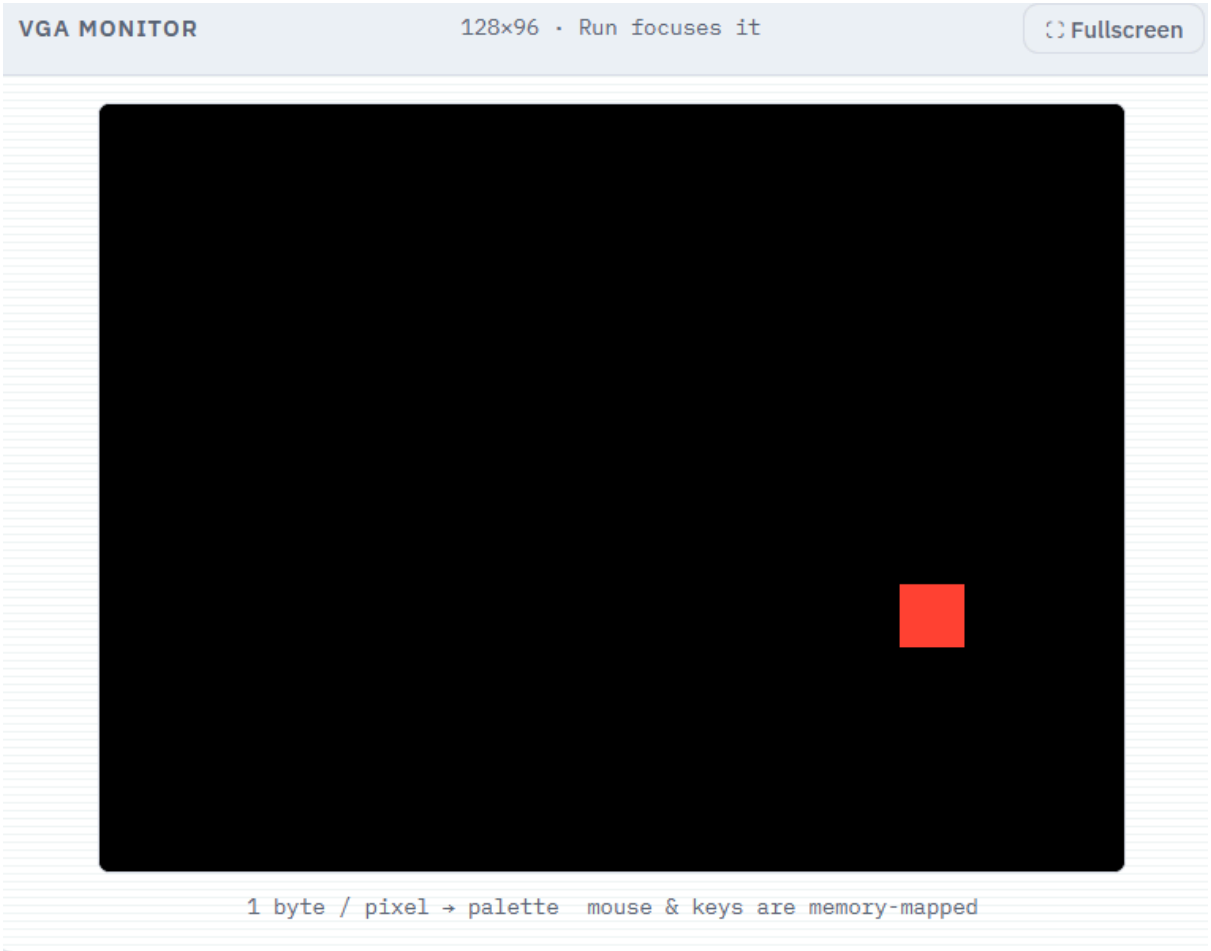
Keyboard control we're using: the C key (ASCII 99, or 67 for uppercase). It will clear the canvas. We read it from the key buffer register at 0x4000002c, since each press should trigger the action once.

Test mark

Our test program is etch_paint_week1_testmark.s. It reads the screen width, clears the screen to black and draws one 8 x 8 red square (palette color 2) with its top left pixel at x = 100, y = 60. Then it writes PRESENT so the square shows up.

If it works, the square should cover x = 100 to 107 and y = 60 to 67 on a black screen, and the console should print the position. In the Playground that puts the left edge of the square about 100/W of the way across the monitor and the top edge about 60/H of the way down.

Result:


VGA MONITOR
128×96 · Run focuses it
Fullscreen
1 byte / pixel → palette mouse & keys are memory-mapped